

参赛项目复现报告

赛题编号: **proj59**

复现日期: 2026 年 6 月 30 日

一、复现目标

本报告用于说明本参赛作品在新设备环境下的可复现性。复现范围覆盖源码检查、环境配置、程序构建、模型推理、trace 采集、结果分析与重复实验汇总等完整流程。

本次复现重点验证以下内容：

- 仓库源码与关键脚本是否完整；
- 项目是否能够从源码构建出 llama-cli；
- 默认模型路径下是否能够完成推理运行；
- trace pipeline 是否能够生成完整的访存行为记录；
- baseline 与优化策略是否能够通过统一脚本复现；
- 对比报告与重复实验汇总是否能够正常生成；
- OS hint 与异步 expert prefetch 等优化机制是否能够稳定触发。

本文中所有路径均以项目根目录为基准。例如：

```
llama.cpp/trace_output/
```

表示项目根目录下的 trace 输出目录。

二、复现环境

本次复现环境如下表所示。

项目	配置
操作系统	Ubuntu 24.04
CPU	13th Gen Intel(R) Core(TM) i5-13500HX
CPU 线程数	20 logical CPUs
内存	15 GiB
Swap	4.0 GiB
GCC	13.3.0
G++	13.3.0
CMake	3.28.3
Python	3.12.3

Python 分析环境使用 .venv 虚拟环境，主要依赖版本如下。

依赖	版本
numpy	2.5.0
pandas	3.0.3
matplotlib	3.11.0

三、仓库版本与源码完整性检查

本次复现使用的提交版本为：

```
a2c6c3905a2d6cba8e9ff83f1701506f53ae35d9
```

最近提交记录如下：

```
a2c6c39 Add async expert prefetch controls
fc82acf Update trace pipeline and scripts
```

```
d8a2715 Add expert cache replacement policies
27c3c5c feat: 添加 LLM 访存 OS hint 优化原型
375ec99 feat: 增强 LLM 访存页驻留观测
```

复现前对关键源码文件和脚本进行了检查，确认以下文件均存在：

```
llama.cpp/src/models/models.h
llama.cpp/tools/mtmd/models/models.h
llama.cpp/trace/compare_trace_runs.py
llama.cpp/trace/run_trace_pipeline.sh
llama.cpp/trace/run_finalist_repeat_matrix.sh
```

上述文件覆盖了构建、trace 采集、结果分析、策略对比和重复实验汇总等关键环节，说明当前仓库具备完整复现所需的源码与脚本结构。

```
git status --short
a2c6c39 (HEAD -> main, origin/main, origin/HEAD) Add async expert prefetch controls
fc82acf Update trace pipeline and scripts
d8a2715 Add expert cache replacement policies
27c3c5c feat: 添加 LLM 访存 OS hint 优化原型
375ec99 feat: 增强 LLM 访存页驻留观测
```

图1 仓库提交版本与工作区状态检查

四、模型文件准备

复现使用默认模型路径：

```
models/Qwen3.5-35B-A3B-Q3_K_M.gguf
```

模型文件放置于项目根目录下的 `models/` 目录中。复现脚本默认读取该路径，因此在新设备上保持相同目录结构即可直接运行。

五、构建过程

在项目根目录下执行以下命令进行构建。首先清理旧构建目录：

```
rm -rf llama.cpp/build
```

执行 CMake 配置：

```
cmake -S llama.cpp -B llama.cpp/build -DLLAMA_MEM_TRACE=ON -DCMAKE_BUILD_TYPE=Release
```

其中 `-DLLAMA_MEM_TRACE=ON` 用于开启本作品所需的内存 `trace` 相关功能。随后编译 `llama-cli`：

```
cmake --build llama.cpp/build --target llama-cli -j"${nproc}"
```

构建完成后执行以下命令验证可执行程序：

```
llama.cpp/build/bin/llama-cli --help | head -n 20
```

终端能够正常输出 `llama-cli` 的参数说明，表明可执行程序已经成功构建。

```
[ 14%] Built target ggml
[ 72%] Built target llama
[ 85%] Built target mtmd
[ 85%] Building CXX object common/CMakeFiles/llama-common.dir/__/license.cpp.o
[ 85%] Linking CXX shared library ../bin/libllama-common.so
[ 98%] Built target llama-common
[100%] Built target server-context
[100%] Linking CXX executable ../bin/llama-cli
[100%] Built target llama-cli
----- common params -----

-h, --help, --usage      print usage and exit
--version                show version and build info
--license                show source code license and dependencies
-cl, --cache-list         show list of models in cache
--completion-bash        print source-able bash completion script for llama.cpp
-t, --threads N          number of CPU threads to use during generation (default: -1)
                        (env: LLAMA_ARG_THREADS)
-tb, --threads-batch N   number of threads to use during batch and prompt processing (default:
                        same as --threads)
-C, --cpu-mask M         CPU affinity mask: arbitrarily long hex. Complements cpu-range
                        (default: "")
-Cr, --cpu-range lo-hi   range of CPUs for affinity. Complements --cpu-mask
--cpu-strict <0|1>       use strict CPU placement (default: 0)
--prio N                 set process/thread priority : low(-1), normal(0), medium(1), high(2),
                        realtime(3) (default: 0)
--poll <0...100>         use polling level to wait for work (0 - no polling, default: 50)
-Cb, --cpu-mask-batch M  CPU affinity mask: arbitrarily long hex. Complements cpu-range-batch
                        (default: same as --cpu-mask)
```

图2 llama-cli 构建成功与可执行程序验证

六、最小链路验证

为了验证模型加载、推理执行、trace 采集和分析报告生成流程，首先运行少量 token 的 smoke_baseline:

```
RUN_NAME=smoke_baseline \
NUM_TOKENS_PREDICT=8 \
LLM_MEM_TRACE_OS_HINTS=0 \
bash llama.cpp/trace/run_trace_pipeline.sh
```

运行完成后，终端输出 Pipeline complete!。对应输出目录为:

```
llama.cpp/trace_output/smoke_baseline/
```

其中 analysis/ 目录成功生成以下文件:

```
01_memory_timeline.png
02_kv_cache.png
03_expert_activation.png
04_tensor_access.png
05_token_latency.png
06_summary_dashboard.png
07_page_residency.png
analysis_report.html
metrics.json
summary.txt
```

该结果表明，推理运行、trace 生成、指标统计和 HTML 报告生成均已跑通。

指标	数值
rss_peak_gb	14.94
total_minor_faults	200,505
total_major_faults	480,805
swap_peak_mb	218.93
prefill_tokens	490
decode_tokens	7
prefill_avg_latency_us	67,155,798.74

decode avg latency us	154,426.83
expert route events	19,328
unique experts global	256

该步骤用于确认完整 pipeline 的可运行性，是后续正式实验的基础。

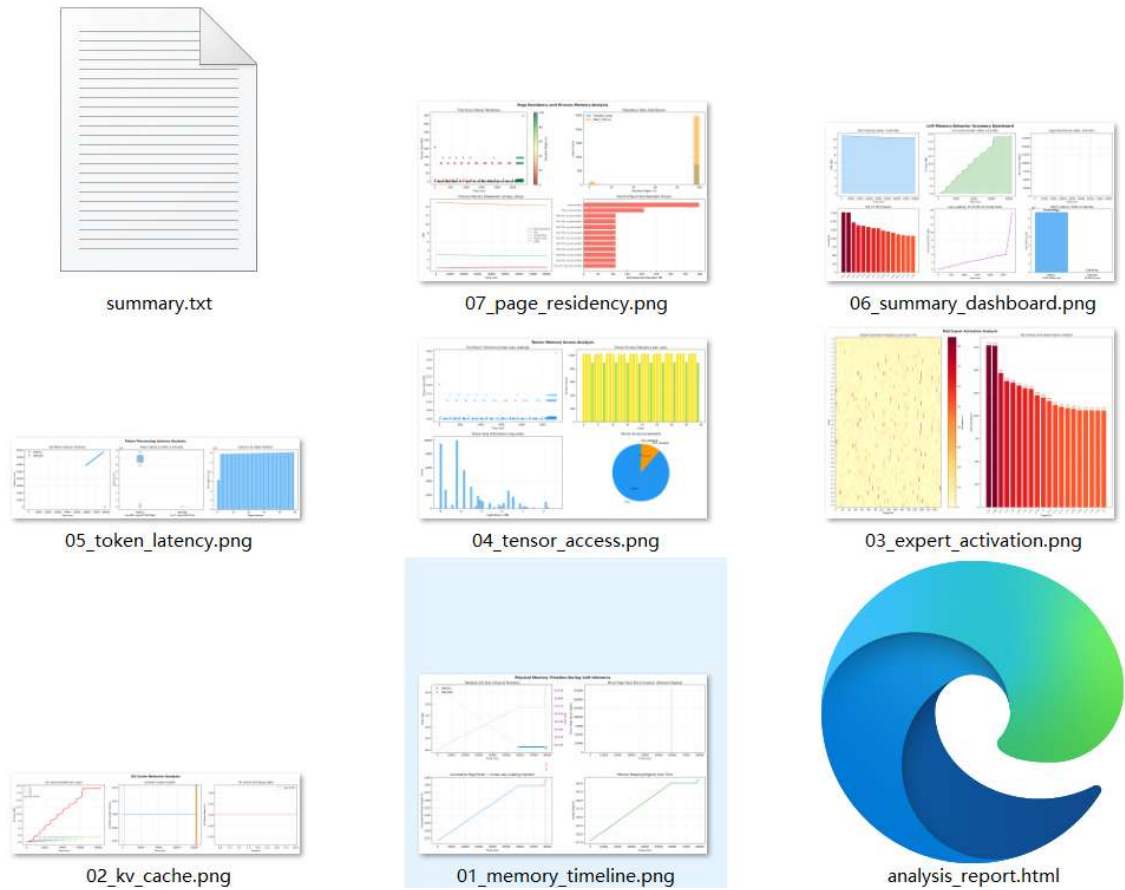


图3 smoke_baseline 分析结果文件生成情况

七、baseline 与 deadline_score 单次对比

完成最小链路验证后，分别运行 baseline 和 deadline_score 策略。baseline 运行命令如下：

```
RUN_NAME=baseline \
NUM_TOKENS_PREDICT=80 \
LLM_MEM_TRACE_OS_HINTS=0 \
LLM_MEM_TRACE_OPT_EXPERT_PREFETCH=0 \
bash llama.cpp/trace/run_trace_pipeline.sh
```

deadline_score 运行命令如下：

```
RUN_NAME=deadline_score \
NUM_TOKENS_PREDICT=80 \
LLM_MEM_TRACE_OS_HINTS=1 \
LLM_MEM_TRACE_OPT_EXPERT_PREFETCH=1 \
LLM_MEM_TRACE_OPT_EXPERT_POLICY=route \
LLM_MEM_TRACE_OPT_EXPERT_PREFETCH_TOPK=0 \
LLM_MEM_TRACE_OPT_EXPERT_ASYNC=1 \
LLM_MEM_TRACE_OPT_EXPERT_ASYNC_QUEUE=131072 \
LLM_MEM_TRACE_OPT_EXPERT_ASYNC_WORKERS=4 \
LLM_MEM_TRACE_OPT_EXPERT_ASYNC_PRIORITY=1 \
```

```
LLM_MEM_TRACE_OPT_EXPERT_ASYNC_PRIORITY_MODE=deadline_score \
LLM_MEM_TRACE_OPT_EXPERT_ASYNC_PRIORITY_HEAP=0 \
bash llama.cpp/trace/run_trace_pipeline.sh
```

两组实验均成功生成 analysis/metrics.json、analysis/summary.txt 和 analysis/analysis_report.html。
单次实验关键指标如下：

Run	Decode avg latency us	Prefill avg latency us	Major faults	Minor faults	RSS peak GB	Swap peak MB	OS hint events	Expert async enqueued
baseline	93,113.17	65,236,352.69	464,621	204,656	14.94	201.33	0	0
deadline_score	114,393.70	69,878,220.88	424,517	214,212	14.94	206.24	100,587	102,162

从单次实验结果看，deadline_score 成功触发 OS hint 与异步 expert prefetch，同时 major faults 从 464,621 降至 424,517，说明该策略能够减少主缺页次数。该结果验证了优化机制的有效触发和访存行为变化。

八、对比报告生成

执行以下命令生成 baseline 与 deadline_score 的对比报告：

```
python3 llama.cpp/trace/compare_trace_runs.py \
--base-dir llama.cpp/trace_output \
--runs baseline deadline_score \
--output-dir llama.cpp/trace_output/compare_baseline_deadline
```

输出目录为 llama.cpp/trace_output/compare_baseline_deadline/，成功生成以下文件：

```
01_os_hint_overview.png
02_os_hint_improvement_heatmap.png
03_fault_delta_timeline.png
comparison_metrics.csv
comparison_report.html
comparison_summary.md
```

Run	Pareto	Major Faults	Minor Faults	Decode us	Prefill us	RSS GiB	Swap MiB	OS Hint Calls
Baseline	yes	464,621	204,656	93,113	65,236,353	14.94	201.33	0
deadline_score	yes	424,517	214,212	114,394	69,878,221	14.94	206.24	100,587

该步骤说明项目能够自动完成实验对比、指标汇总、图表生成和 HTML 报告生成。

```
cat llama.cpp/trace_output/compare_baseline_deadline/comparison_summary.md
total 352K
-rwxrwxrwx 1 zhang zhang 144K Jun 30 12:56 01_os_hint_overview.png
-rwxrwxrwx 1 zhang zhang 63K Jun 30 12:56 02_os_hint_improvement_heatmap.png
-rwxrwxrwx 1 zhang zhang 129K Jun 30 12:56 03_fault_delta_timeline.png
-rwxrwxrwx 1 zhang zhang 3.8K Jun 30 12:56 comparison_metrics.csv
-rwxrwxrwx 1 zhang zhang 2.7K Jun 30 12:56 comparison_report.html
-rwxrwxrwx 1 zhang zhang 874 Jun 30 12:56 comparison_summary.md
# OS Hint 优化对比摘要

本摘要基于同一 prompt、同一模型、同一 trace pipeline 的单次消融实验。当前结论表示效果幅度明显，严格统计显著性仍需要多次重复实验。

## 核心结论

- Pareto 最优实验组（同时考虑 decode latency、major faults、RSS、swap 和 OS hint 系统调用）：Baseline、deadline_score。

## 指标表

| Run | Pareto | Major Faults | Minor Faults | Decode us | Prefill us | Residency % | RSS GiB | Swap MiB | OS Hint Calls |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Baseline | yes | 464,621 | 204,656 | 93,113 | 65,236,353 | 77.76 | 14.94 | 201.33 | 0 |
| deadline_score | yes | 424,517 | 214,212 | 114,394 | 69,878,221 | 77.49 | 14.94 | 206.24 | 100,587 |
```

图 4 baseline 与 deadline_score 对比报告生成结果

九、重复实验矩阵

为了进一步验证不同策略在重复运行下的表现，执行最终重复实验矩阵。矩阵包含四组策略：

```
baseline
expert_prefetch
deadline_score
decode_ttl1
```

运行命令如下：

```
RUN_REPEAT_MATRIX_EXECUTE=1 \
RUN_PREFIX=contest_finalist \
REPEAT_COUNT=3 \
bash llama.cpp/trace/run_finalist_repeat_matrix.sh
```

重复实验汇总结果位于 llama.cpp/trace_output/contest_runs/repeat_summary_fixed/，其中包含 repeat_summary.md 和 repeat_summary.csv。本报告采用 repeat_summary_fixed/repeat_summary.md 作为最终重复实验汇总依据。该汇总基于多次有效运行结果聚合得到，能够稳定反映各策略在当前环境下的表现。

十、重复实验结果

1. Decode 平均延迟

Group	N	Mean us	CV %	Improvement vs Baseline %
baseline	4	101,204	1.42	0.00
expert_prefetch	3	117,314	12.73	-15.92
deadline_score	3	124,019	1.11	-22.54
decode_ttl1	3	118,639	3.13	-17.23

2. Prefill 平均延迟

Group	N	Mean us	CV %	Improvement vs Baseline %
baseline	4	73,302,413	1.70	0.00
expert_prefetch	3	96,967,760	18.04	-32.28
deadline_score	3	66,247,969	1.77	9.62
decode_ttl1	3	64,990,167	3.41	11.34

3. Major Faults

Group	N	Mean	CV %	Improvement vs Baseline %
baseline	4	490,334	1.11	0.00
expert_prefetch	3	406,237	5.07	17.15
deadline_score	3	423,484	1.38	13.63
decode_ttl1	3	410,474	4.16	16.29

4. OS hint 事件

Group	N	Mean
baseline	4	0

expert prefetch	3	101,099
deadline score	3	100,558
decode ttl	3	83,847

5. 异步 expert prefetch

Group	N	expert_async_enqueued	expert_async_max_queue_depth
baseline	4	0	0
expert prefetch	3	0	0
deadline score	3	102,162	753.33
decode ttl	3	85,353	753.67

```
# Repeat Run Summary

This summary aggregates per-run `analysis/metrics.json` files.

## decode_avg_latency_us

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | --- | --- | --- | --- | --- | --- | --- |
| baseline_clean | 4 | 101,204 | 1,433 | 99,324 | 102,457 | 1.4155 | 0.0000 |
| expert_prefetch | 3 | 117,314 | 14,936 | 107,471 | 134,500 | 12.73 | -15.92 |
| deadline_score | 3 | 124,019 | 1,373 | 122,720 | 125,456 | 1.1073 | -22.54 |
| decode_ttl | 3 | 118,639 | 3,711 | 114,360 | 120,962 | 3.1276 | -17.23 |

## prefill_avg_latency_us

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | --- | --- | --- | --- | --- | --- | --- |
| baseline_clean | 4 | 73,302,413 | 1,245,056 | 71,740,660 | 74,780,619 | 1.6985 | 0.0000 |
| expert_prefetch | 3 | 96,967,760 | 17,492,075 | 85,988,658 | 117,139,517 | 18.04 | -32.28 |
| deadline_score | 3 | 66,247,969 | 1,175,835 | 65,432,818 | 67,595,882 | 1.7749 | 9.6238 |
| decode_ttl | 3 | 64,990,167 | 2,212,947 | 62,666,825 | 67,073,086 | 3.4050 | 11.34 |

## total_major_faults

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | --- | --- | --- | --- | --- | --- | --- |
| baseline_clean | 4 | 490,334 | 5,433 | 484,701 | 496,073 | 1.1079 | 0.0000 |
| expert_prefetch | 3 | 406,237 | 20,599 | 383,356 | 423,305 | 5.0707 | 17.15 |
| deadline_score | 3 | 423,484 | 5,833 | 419,867 | 430,213 | 1.3773 | 13.63 |
| decode_ttl | 3 | 410,474 | 17,090 | 390,785 | 421,467 | 4.1634 | 16.29 |

## total_minor_faults

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | --- | --- | --- | --- | --- | --- | --- |
| baseline_clean | 4 | 212,733 | 2,145 | 210,311 | 215,404 | 1.0081 | 0.0000 |
| expert_prefetch | 3 | 212,426 | 10,076 | 200,987 | 219,988 | 4.7435 | 0.1444 |
| deadline_score | 3 | 216,744 | 2,028 | 215,042 | 218,987 | 0.9354 | -1.8852 |
| decode_ttl | 3 | 209,204 | 5,855 | 202,480 | 213,178 | 2.7989 | 1.6588 |
```

图5 重复实验延迟与缺页指标汇总

```
## rss_peak_gb

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 14.93 | 0.0117 | 14.91 | 14.94 | 0.0786 | 0.0000 |
| expert_prefetch | 3 | 14.93 | 0.0051 | 14.93 | 14.94 | 0.0344 | -0.0293 |
| deadline_score | 3 | 14.94 | 0.0079 | 14.93 | 14.94 | 0.0527 | -0.0678 |
| decode_ttl1 | 3 | 14.94 | 0.0044 | 14.93 | 14.94 | 0.0293 | -0.0671 |

## rss_avg_gb

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 14.42 | 0.0145 | 14.40 | 14.43 | 0.1007 | 0.0000 |
| expert_prefetch | 3 | 14.49 | 0.0236 | 14.47 | 14.51 | 0.1631 | -0.4852 |
| deadline_score | 3 | 14.47 | 0.0100 | 14.46 | 14.48 | 0.0694 | -0.3883 |
| decode_ttl1 | 3 | 14.49 | 0.0293 | 14.46 | 14.52 | 0.2024 | -0.4955 |

## swap_peak_mb

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 227.37 | 17.29 | 205.78 | 243.04 | 7.6034 | 0.0000 |
| expert_prefetch | 3 | 203.97 | 19.26 | 182.06 | 218.25 | 9.4448 | 10.29 |
| deadline_score | 3 | 214.52 | 11.39 | 201.38 | 221.20 | 5.3087 | 5.6512 |
| decode_ttl1 | 3 | 204.61 | 21.12 | 180.32 | 218.70 | 10.32 | 10.01 |

## os_hint_events

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| expert_prefetch | 3 | 101,099 | 91.76 | 101,026 | 101,202 | 0.0908 | |
| deadline_score | 3 | 100,558 | 61.61 | 100,511 | 100,628 | 0.0613 | |
| decode_ttl1 | 3 | 83,847 | 79.75 | 83,786 | 83,937 | 0.0951 | |
```

图6 重复实验内存占用与 OS hint 指标汇总

```
## os_hint_advised_mb

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| expert_prefetch | 3 | 46,059 | 44.52 | 46,023 | 46,109 | 0.0967 | |
| deadline_score | 3 | 45,820 | 27.99 | 45,798 | 45,852 | 0.0611 | |
| decode_ttl1 | 3 | 38,219 | 40.81 | 38,191 | 38,266 | 0.1068 | |

## expert_async_enqueued

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| expert_prefetch | 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| deadline_score | 3 | 102,162 | 0.0000 | 102,162 | 102,162 | 0.0000 | |
| decode_ttl1 | 3 | 85,353 | 0.0000 | 85,353 | 85,353 | 0.0000 | |

## expert_async_fallback

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| expert_prefetch | 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| deadline_score | 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| decode_ttl1 | 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |

## expert_async_max_queue_depth

| Group | N | Mean | Std | Min | Max | CV % | Improvement vs Baseline % |
| --- | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
| baseline_clean | 4 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| expert_prefetch | 3 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 | |
| deadline_score | 3 | 753.33 | 0.5774 | 753.00 | 754.00 | 0.0766 | |
| decode_ttl1 | 3 | 753.67 | 0.5774 | 753.00 | 754.00 | 0.0766 | |
```

图7 异步 expert prefetch 运行状态汇总

十一、结果分析

重复实验结果显示，优化策略能够稳定触发，并在访存行为上产生明显影响。

首先，在 OS hint 事件方面，baseline 组未产生 OS hint 调用，而 expert_prefetch、deadline_score 和 decode_ttl1 均产生了大量 OS hint 事件。其中 expert_prefetch 平均产生 101,099 次 OS hint，deadline_score 平均产生 100,558 次，decode_ttl1 平均产生 83,847 次。这说明 OS hint 相关路径在重复实验中稳定生效。

其次，在异步 expert prefetch 方面，deadline_score 和 decode_ttl1 均产生了稳定的异步 expert 入队事件。其中 deadline_score 的 expert_async_enqueued 为 102,162，decode_ttl1 为 85,353，且队列峰值深度稳定在约 753 左右。这说明异步 expert prefetch 机制已经被成功触发。

在主缺页次数方面，三种优化策略均相较 baseline 有下降：expert_prefetch 下降 17.15%，deadline_score 下降 13.63%，decode_ttl1 下降 16.29%。这表明优化策略能够有效减少 major faults，对模型推理过程中的访存行为具有改善作用。

在 prefill 阶段，deadline_score 和 decode_ttl1 相较 baseline 分别取得 9.62% 和 11.34% 的改善，说明基于访问提示和 expert 预取的策略能够在 prompt processing 阶段带来一定收益。

在 decode 阶段，优化策略的平均延迟高于 baseline，说明当前实现中 OS hint 调用、预取调度和异步队列管理仍会引入一定额外开销。该现象符合内存优化策略常见的工程权衡：减少主缺页次数并不必然立即转化为所有阶段的端到端延迟收益，仍需要进一步结合预取粒度和调度策略进行优化。

综合来看，本次复现证明了项目能够在新环境中从源码完成构建；trace 采集、分析和报告生成流程完整可用；baseline 与优化策略能够通过统一脚本复现；OS hint 和异步 expert prefetch 机制能够稳定触发；优化策略能够明显降低 major faults；deadline_score 与 decode_ttl1 在 prefill 阶段体现出一定收益；decode 阶段仍存在进一步优化空间。

十二、复现产物

本次复现生成的主要产物如下：

```
llama.cpp/build/bin/llama-cli

llama.cpp/trace_output/smoke_baseline/analysis/analysis_report.html
llama.cpp/trace_output/smoke_baseline/analysis/metrics.json
llama.cpp/trace_output/smoke_baseline/analysis/summary.txt

llama.cpp/trace_output/baseline/analysis/metrics.json
llama.cpp/trace_output/deadline_score/analysis/metrics.json

llama.cpp/trace_output/compare_baseline_deadline/comparison_report.html
llama.cpp/trace_output/compare_baseline_deadline/comparison_summary.md
llama.cpp/trace_output/compare_baseline_deadline/comparison_metrics.csv

llama.cpp/trace_output/contest_runs/repeat_summary_fixed/repeat_summary.md
llama.cpp/trace_output/contest_runs/repeat_summary_fixed/repeat_summary.csv
```

上述文件能够作为本次复现成功的直接证据。

十三、附图说明

图号	图名	内容
图 1	仓库提交版本与工作区状态检查	git log、git status 与提交版本信息
图 2	llama-cli 构建成功与可执行程序验证	构建完成输出与 llama-cli --help 输出
图 3	smoke_baseline 分析结果文件生成情况	analysis 目录下的图表、summary.txt 和 analysis_report.html
图 4	baseline 与 deadline_score 对比报告生成结果	compare_baseline_deadline 目录与 comparison_summary.md
图 5	重复实验延迟与缺页指标汇总	decode、prefill、major faults 和 minor faults
图 6	重复实验内存占用与 OS hint 指标汇总	RSS、swap 与 OS hint events
图 7	异步 expert prefetch 运行状态汇总	expert_async_enqueued 与 expert_async_max_queue_depth

十四、复现结论

本次复现完成了从源码检查、环境配置、模型准备、程序构建，到最小链路验证、单次策略对比和重复实验汇总的完整流程。

复现结果表明，当前版本能够稳定构建并运行 llama-cli，能够生成完整 trace、分析报告和对比报告；同时，OS hint 与异步 expert prefetch 等优化机制能够在实验中稳定触发。重复实验结果显示，优化策略能够降低 major faults，并在 prefill 阶段体现一定收益。

因此，本参赛作品具备完整的工程复现能力，能够在新设备上按照给定流程完成构建、运行、trace 采集和结果分析。